A-Commerce — Technical Architecture Documentation

System Name:	A-Commerce Production Platform	**Target Environment:**	PHP 8.x / MySQL 8.0 / Apache
Documentation Version:	v1.0.0 (Release)	**Security Review Status:**	**Passed (Built-in Hardening)**
Architecture Model:	Procedural Modular Model-View-Controller Variant	**Date Generated:**	June 9, 2026

1. Executive Project Overview

A-Commerce is a high-performance, ultra-lightweight, native PHP/MySQL electronic commerce platform designed to support secure product listings, state-backed session shopping carts, asynchronous checkout operations, and an integrated administrative intelligence dashboard.

The software design rejects bloated framework dependencies in favor of optimized, procedural scripts powered by robust object-oriented abstractions where critical (such as the PHP Data Objects engine). This architectural choice yields near-zero execution overhead, minimal latency profiles, and predictable deployment patterns suitable for high-density horizontal container scaling.

2. Global Application Lifecycle & Bootstrap Architecture

The root structural lifecycle is tied tightly to the global dependency injection component: `db.php`. Every dynamic template initiates execution by pulling this configuration layer. It serves three distinct architectural duties: directive runtime adjustment, stateful session initialization, and database connection pooling.

2.1 Runtime Optimization Directives

To eliminate standard configuration risks and prevent Session Hijacking or Cross-Site Scripting (XSS) Session theft, the system overrides default runtime initialization configurations via `ini_set()` before opening global scopes:

- `session.use_strict_mode = 1`: Prevents Session Fixation attacks by rejecting uninitialized session identifiers explicitly requested by arbitrary clients.
- `session.use_only_cookies = 1`: Mitigates URI-based leakage vectors by enforcing that data keys never enter string requests.

- `session.use_trans_sid = 0`: Disables completely the transparent rewriting of URLs with raw session tokens.
- `session.cookie_httponly = 1`: Sets the HTTP-Only state on the client cookie, blocking access via JavaScript `document.cookie` scripts.
- `session.cookie_samesite = Lax`: Directs modern user agents to withhold credentials during cross-site requests, providing baseline Cross-Site Request Forgery (CSRF) protection.

3. Database Design & Connection Topology

The platform relies on raw, non-emulated parameterized queries using the PDO abstract interface. Connection management relies on a clean Singleton/Static optimization variable pattern within the `getPDO()` routine to prevent resource-exhaustion under high concurrency.

```
function getPDO(): PDO {
    static $pdo = null;
    if ($pdo !== null) return $pdo;

    $dsn = sprintf('mysql:host=%s;dbname=%s;charset=%s', DB_HOST, DB_NAME, DB_CHARSET);
    $options = [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false, // Enforces true server-side parameter binding
    ];
    // Connection initialization block...
}
```

3.1 Logical Schema Specifications

The implied application datastore maps directly across four core dynamic normalization models:

Entity	Attributes & Constraints	Relationships
users	`id (INT, PK, AI)`, `name (VARCHAR)`, `email (VARCHAR, UNIQUE)`, `password (VARCHAR)`, `is_admin (BOOLEAN)`	One-to-Many with `orders`
products	`id (INT, PK, AI)`, `title (VARCHAR)`, `description (TEXT)`, `price (DECIMAL(10,2))`, `image_url (VARCHAR)`, `stock (INT)`, `created_at (TIMESTAMP)`	One-to-Many with `order_items`
orders	`id (INT, PK, AI)`, `user_id (INT, FK)`, `total (DECIMAL(10,2))`, `created_at (TIMESTAMP)`	Many-to-One with `users`; One-to-Many with `order_items`
order_items	`id (INT, PK, AI)`, `order_id (INT, FK CASCADE)`, `product_id (INT, FK)`, `quantity (INT)`, `price (DECIMAL(10,2))`	Composite link table connecting orders and products

4. Granular Component Directory & Dynamic Analysis

4.1 User-Facing Storefront (`index.php`)

Acts as the main operational gateway and item discovery grid. It requires a valid session to process and features a dual-mode interaction schema: standard dynamic structural layout generation for standard views, and a native JSON API endpoint for incoming transactional requests.

When processing an `add_to_cart` transaction, the controller switches its output stream buffer using explicit HTTP header modification (`Content-Type: application/json`). It validates the parameter bindings, checks physical inventory boundaries directly from isolated database tuples, and increments the in-memory item mapping structure inside `$_SESSION['cart']`.

4.2 Session Shopping Matrix (`cart.php`)

Manages client checkout pre-flight pipelines entirely in-memory before relational ledger conversion. It supports individual item point deletions, manual quantities recalculations, state purges, and single-click checkout simulation.

To prevent cross-thread synchronization issues, checking out clears out the associative multi-dimensional arrays inside `$_SESSION['cart']` upon successful delivery acknowledgment and sets structural flash states.

4.3 Cryptographic Authentication Hubs (`login.php` & `login-handler.php`)

The authentication pipeline utilizes hardened defense-in-depth patterns. When standard login payloads arrive via POST, they pass through explicit `FILTER_VALIDATE_EMAIL` filters. User lookup relies on a parameterized hash match lookup.

Password compliance verification is offloaded to `password_verify()`, enforcing protection against runtime timing analysis. If a lookups fails or a credential mismatch occurs, the controller injects a microsecond-level process execution penalty via `usleep(200000)` to throttle horizontal dictionary or brute-force scanning vectors. Upon successful authorization, a mandatory `session_regenerate_id(true)` wipes old session pointers, neutralising session fixation threats.

4.4 Administrative Intelligence Center (`admin-dashboard.php` & `admin-handler.php`)

Restricted via administrative guards (`requireAdmin()`), this center handles complex operational analytics reporting alongside catalog data mutation inputs. The interface handles analytical monitoring queries through deep tabular relational aggregates:

```
// Dynamic Analytical Aggregate Query Example
SELECT o.id, o.user_id, u.name as user_name, o.total, o.created_at, COUNT(oi.id) as item_coun
FROM orders o
LEFT JOIN users u ON o.user_id = u.id
LEFT JOIN order_items oi ON o.id = oi.order_id
GROUP BY o.id
ORDER BY o.created_at DESC LIMIT 10
```

The handler file manages three discrete mutation workflows: `add_product`, `delete_product`, and `update_stock`. Each transactional pipeline executes deep parameter cleaning, rejecting invalid pricing values via floating-point mapping checks and stock boundaries via signed integer validations.

4.5 Session Destruction Strategy (`logout.php`)

Ensures proper, zero-remanence state termination. The file programmatically flushes active memory bounds (`$_SESSION = []`), forces tracking cookie expiration backwards into historical epoch space (`time() - 42000`) against specific local parameters, and calls `session_destroy()` to force garbage collection routines on the physical hosting server.

5. Global Security Matrix & Interceptor Framework

The platform maintains an aggressive security posture through structured software layers:

5.1 Multi-Tier Cryptographic Token Verification (CSRF)

State modification operations (POST) require validation against an alphanumeric pseudo-random sequence generated via `bin2hex(random_bytes(32))` stored directly in protected server memory. Verification occurs using `hash_equals()`, guaranteeing constant-time execution and preventing side-channel statistical runtime discovery.

5.2 Input Mitigation and Output Serialization Architecture (XSS)

To intercept malicious script payloads injected via product metadata or user strings, the platform runs output rendering through strict escape boundaries: `htmlspecialchars($var, ENT_QUOTES, 'UTF-8')`. This breaks character sequences like `<`, `>`, and single/double quotation boundaries into harmless entity literals before they hit user layout runtimes.

5.3 Granular Request Access Interceptors

Access mapping rules run procedurally via interceptor guards:

Guard Routine	Internal Execution Condition	Failure Action Vector
`requireLogin()`	Checks `isset($_SESSION['user_id'])`	Flashes warning; issues HTTP redirect header to `login.php`
`requireAdmin()`	Checks `$_SESSION['is_admin'] === true`	Flashes severe violation; issues HTTP redirect header to `index.php`